Spatial and Temporal Locality, Caches and Memory hierarchy

Sparsh Mittal

Courtesy for a few slides: MK Publishers, and others

Real-life example of caches

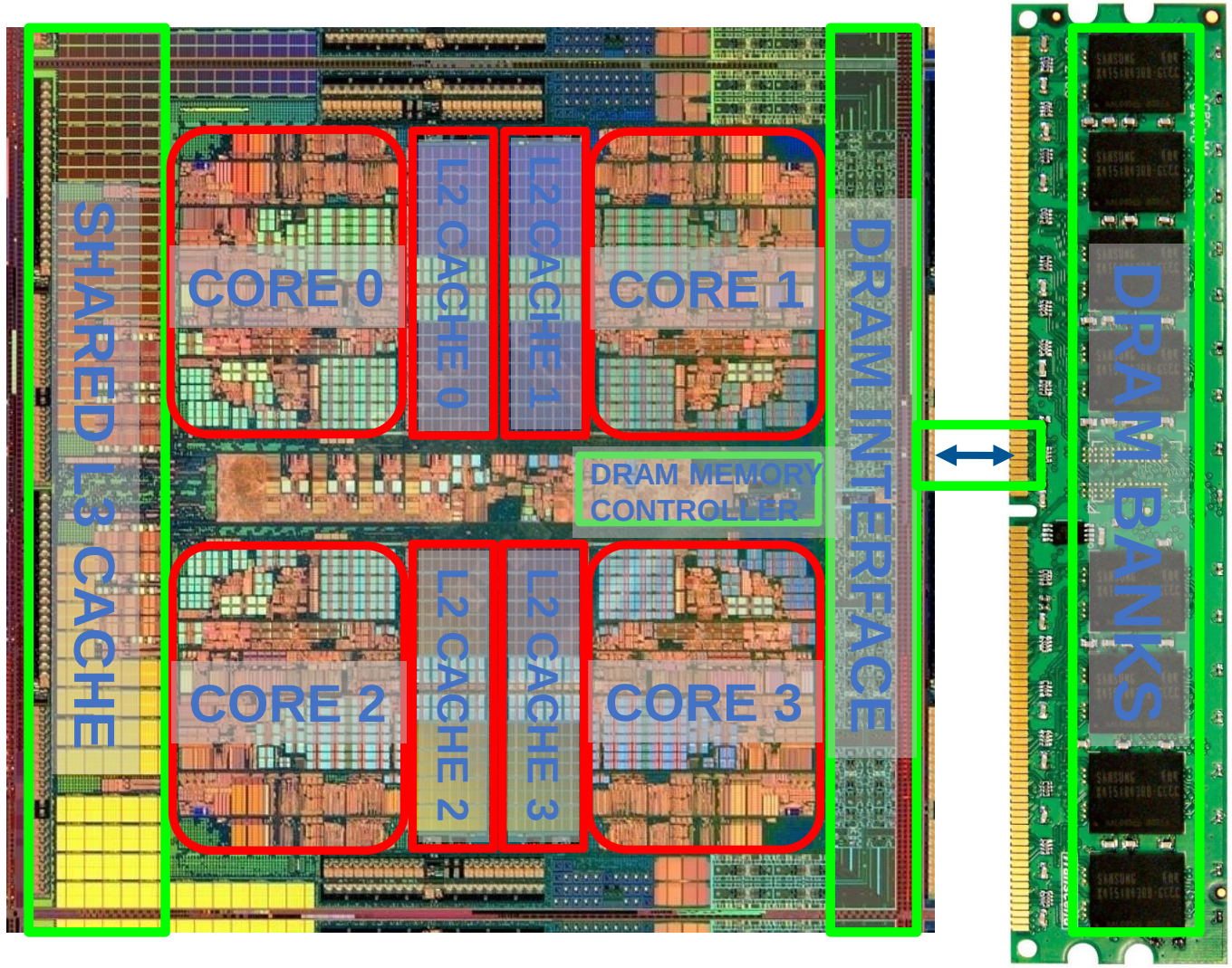
Internet browsers store recently accessed websites, since they are likely to be accessed again.

We bookmark frequently visited websites

An encyclopedia has all the concepts. A handbook or a pocket dictionary contains selected rules/words

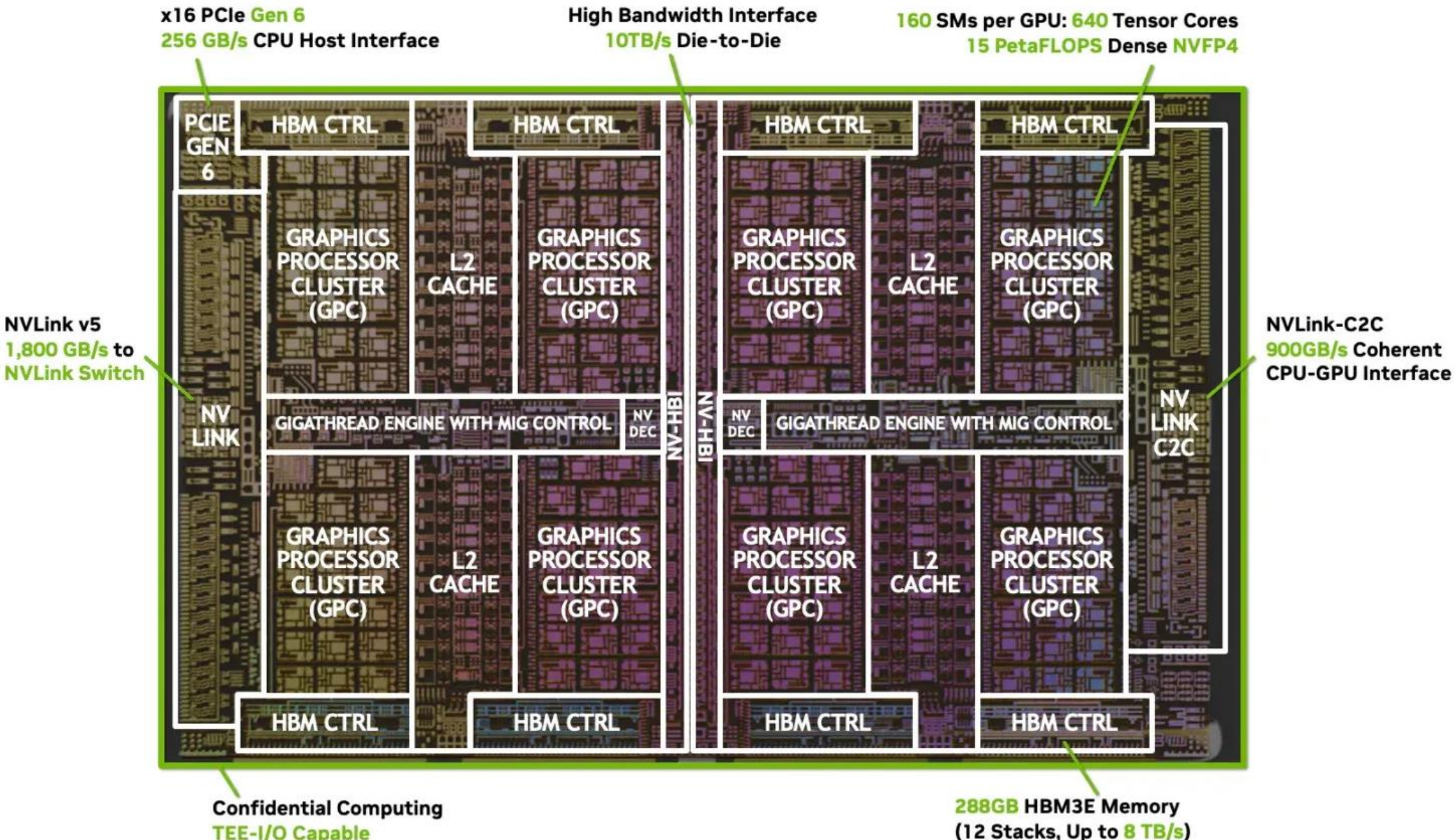
A cheat-sheet has most frequently used formulae.

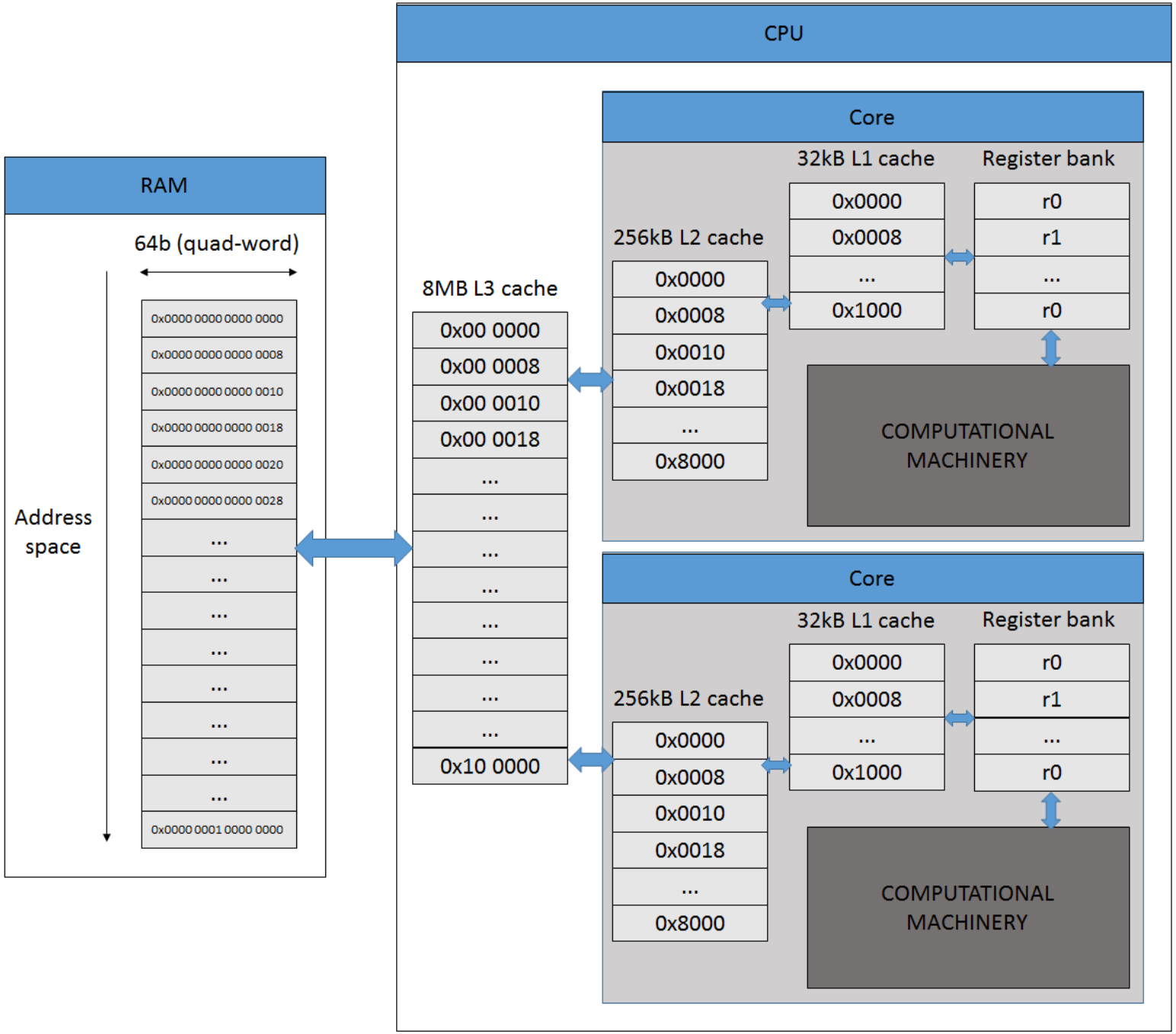
Examples of processor chip



L1, L2, L3 cache => L1 is closest and L3 is farthest from core

NVIDIA Blackwell Ultra GPU




CPU
RAM
64b (quad-word)
Address space
0x0000 0000 0000 0000
0x0000 0000 0000 0008
0x0000 0000 0000 0010
0x0000 0000 0000 0018
0x0000 0000 0000 0020
0x0000 0000 0000 0028
...
0x0000 0001 0000 0000
8MB L3 cache
0x00 0000
0x00 0008
0x00 0010
0x00 0018
0x10 0000
Core
32kB L1 cache
Register bank
256kB L2 cache
0x0000
0x0008
0x1000
r0
r1
0x0000
0x0008
0x0010
0x0018
0x8000
COMPUTATIONAL MACHINERY
Core
32kB L1 cache
Register bank
256kB L2 cache
0x0000
0x0008
0x1000
r0
r1
0x0000
0x0008
0x0010
0x0018
0x8000
COMPUTATIONAL MACHINERY

Principle of Locality (of address access)

Programs access a small proportion of their address space at any time

Temporal locality

- Items accessed recently are likely to be accessed again soon
  e.g., instructions in a loop

Spatial locality

- Items near those accessed recently are likely to be accessed soon
  E.g., sequential instruction access, array data

Finding locality at address-access level

Which of these patterns show which type of locality:

Pattern1: 2040, 30, 409, 333, 891, 801, 302, ...

Pattern2: 201, 945, 201, 201, 120, 201, 201, 201, ...

Pattern3: 658, 659, 660, 661, 662, 663, 664, ...

Pattern4: 24, 25, 24, 26, 25, 27, 24, 28, 29, 25,....

Answer: P1: none P2: Temporal P3: spatial P4: both

Finding locality at source-code level

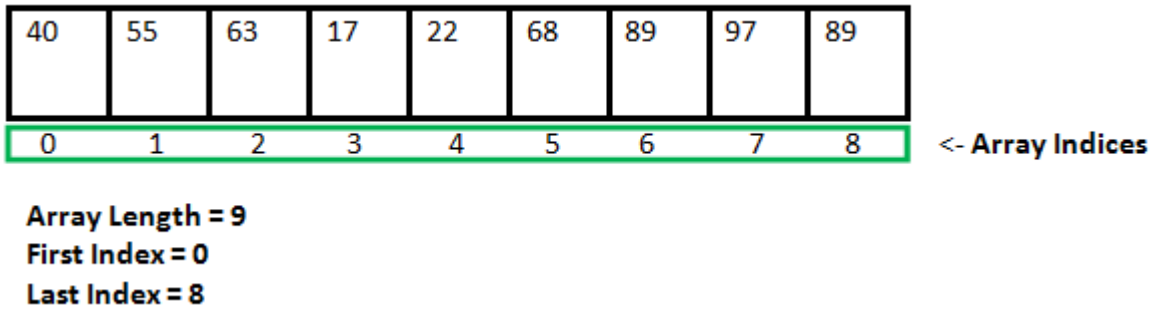
```
sum = 0;
 for (i = 0; i < arr.length; i++)
   sum += arr[i];
 return sum;
```

- Answer: sum and i have temporal locality
- Access to Arr elements shows spatial locality
- Instruction corresponding to **+=** will be executed repeatedly, showing temporal locality.

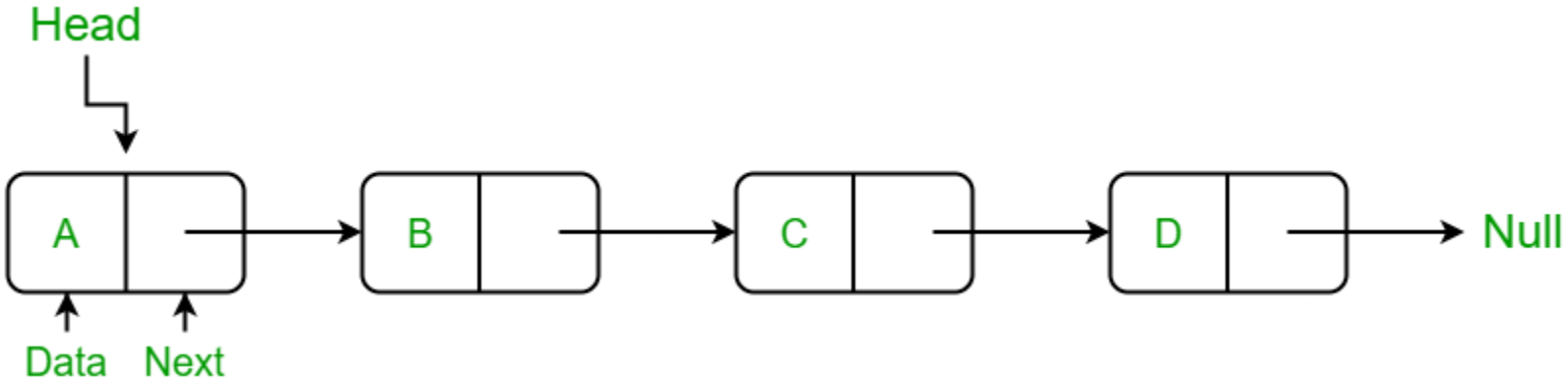
Question on spatial locality [1 of 2]

We have multiple numbers which can be stored in array or dynamic linked-list.

Array



Linked list



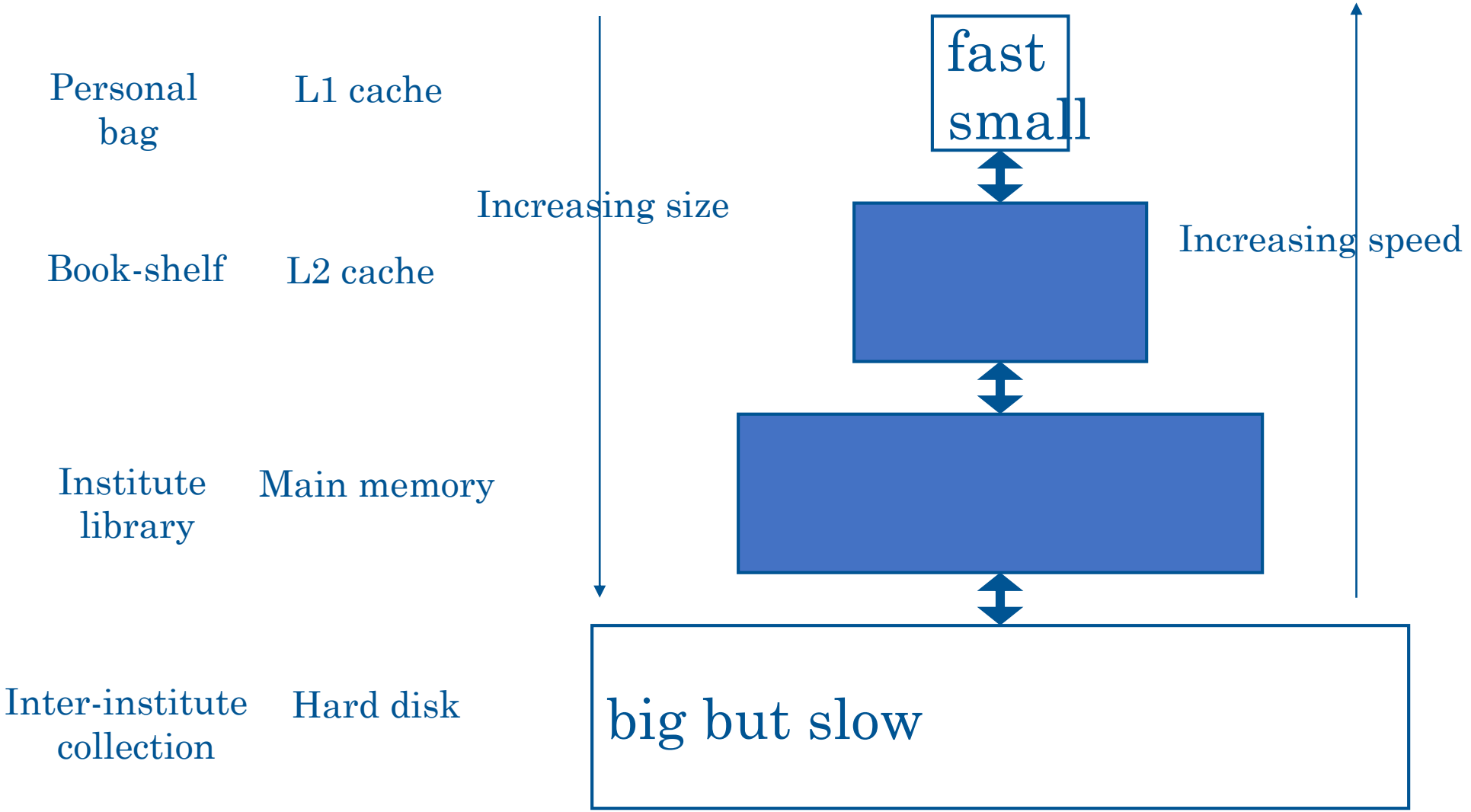
Question on spatial locality [2 of 2]

If an application accesses these numbers sequentially, which of the two data-structures are able to benefit from the spatial locality present in the application?

Answer: Array.

Explanation: The array elements are stored contiguously whereas linked-list elements are not stored near each other.

Real-life example of caches



What is a cache in a processor

- Main memory accesses take huge latency and energy
- Cache, also called cache memory, stores frequently accessed data
- If items stored in cache are again accessed, they are supplied from cache.
- This avoids the penalty of accessing main memory.
- By observing the past, we try to predict the future.

How locality and cache are related

- Based on temporal locality: programs will access a few items/variables in a short window. We can store them in cache.
- Based on spatial locality: if an array element Arr[i] has been accessed, we can fetch Arr[i+1] to Arr[i+k] elements from main memory to cache. These elements are likely to get accessed very soon.
- Thus, principles of locality allow us to design cache
- If an application has no locality, caches will not be effective for it.

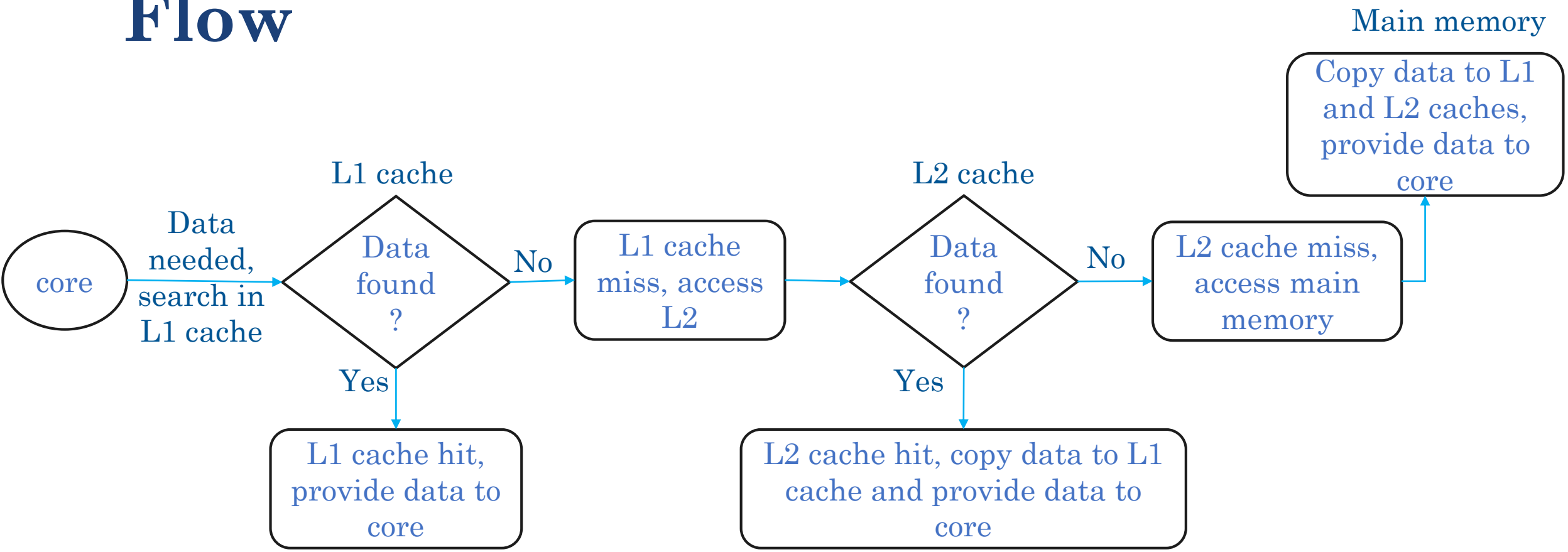
Defining cache block (line)

- Cache block is also called cache line
- It is the minimum unit of data transfer
- Example: If I go to library, the minimum I can issue is a book. I cannot issue a single page
- Similarly, if block size is 64B, then even if a processor accesses one byte, it will get 64B of data from cache.

Defining Cache Hit and Cache Miss

- When processor accesses L1 cache, if the accessed data is present in L1 cache, it is called an **hit in L1 cache**.
- Otherwise, it is called a **miss in L1 cache**. Then, processor will search the data in L2 cache.
- Now, if data is present in L2 cache, it is called a hit in L2 cache
- Otherwise, it is a miss in L2 cache. The processor will search the data in next lower level (L3 cache or main memory)
- We assume that main memory has no misses.

Flow


Main memory
Copy data to L1 and L2 caches, provide data to core
L1 cache
L2 cache
core
Data needed, search in L1 cache
Data found ?
No
L1 cache miss, access L2
Data found ?
No
L2 cache miss, access main memory
Yes
Yes
L1 cache hit, provide data to core
L2 cache hit, copy data to L1 cache and provide data to core

Defining Hit Rate and Miss Rate

- A cache gets “A” accesses, of which “M” accesses miss
- Miss Rate of this cache = M/A = Misses/Accesses
- Hit Rate of this cache = (A-M)/A = Hits/Accesses

- Example: L1 cache gets 10,000 access and has 9500 hits.
- Miss rate is 5%. Hit rate is 95%

Miss penalty and I-cache/D-cache

Miss penalty: On L1 miss, we get data from L2 cache. The time taken in this is called L1 miss penalty.

On L2 miss, we get data from main memory (assuming no L3 cache). This latency is L2 miss penalty

Since instruction and data accesses show different properties, we design separate caches for them. They are called I-cache and D-cache, respectively

Memory Hierarchy (also called cache hierarchy)

Candidates for designing memory

Static RAM (SRAM)

Dynamic RAM (DRAM)

Magnetic disk

Latency increases

Density increases

Cost/GB reduces

No single memory technology has all the desired characteristics.

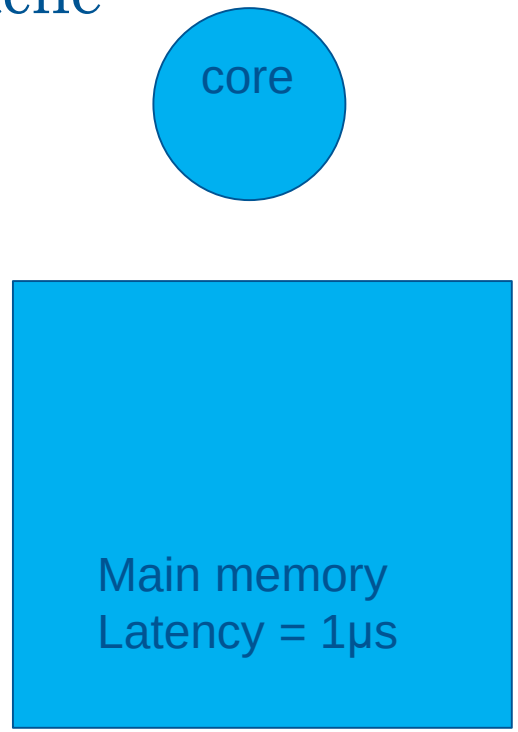
How to design an ideal memory?

- We want to design a memory, which has
  - Very low latency (to provide data quickly)
  - Very high capacity (to keep miss-rate low)
  - Low price

- How to approach the characteristic of such a memory?
- Answer: use a memory-hierarchy

Approaching ideal memory through memory "hierarchy"?

Case 1: no cache

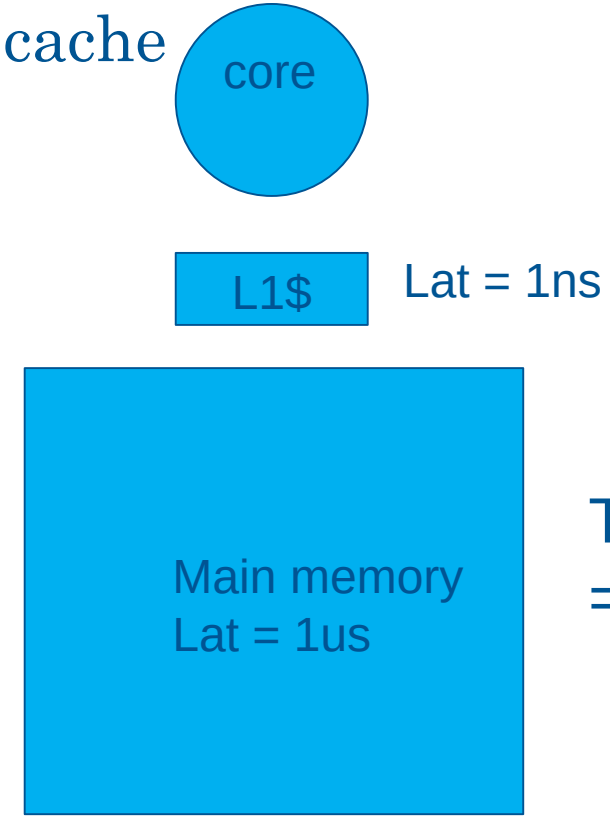


Processor core sends 1 million requests

Total time incurred in serving 1M requests
= 1 million * 1micro-second = 1 sec

Approaching ideal memory through memory "hierarchy"?

Case 2: Use cache

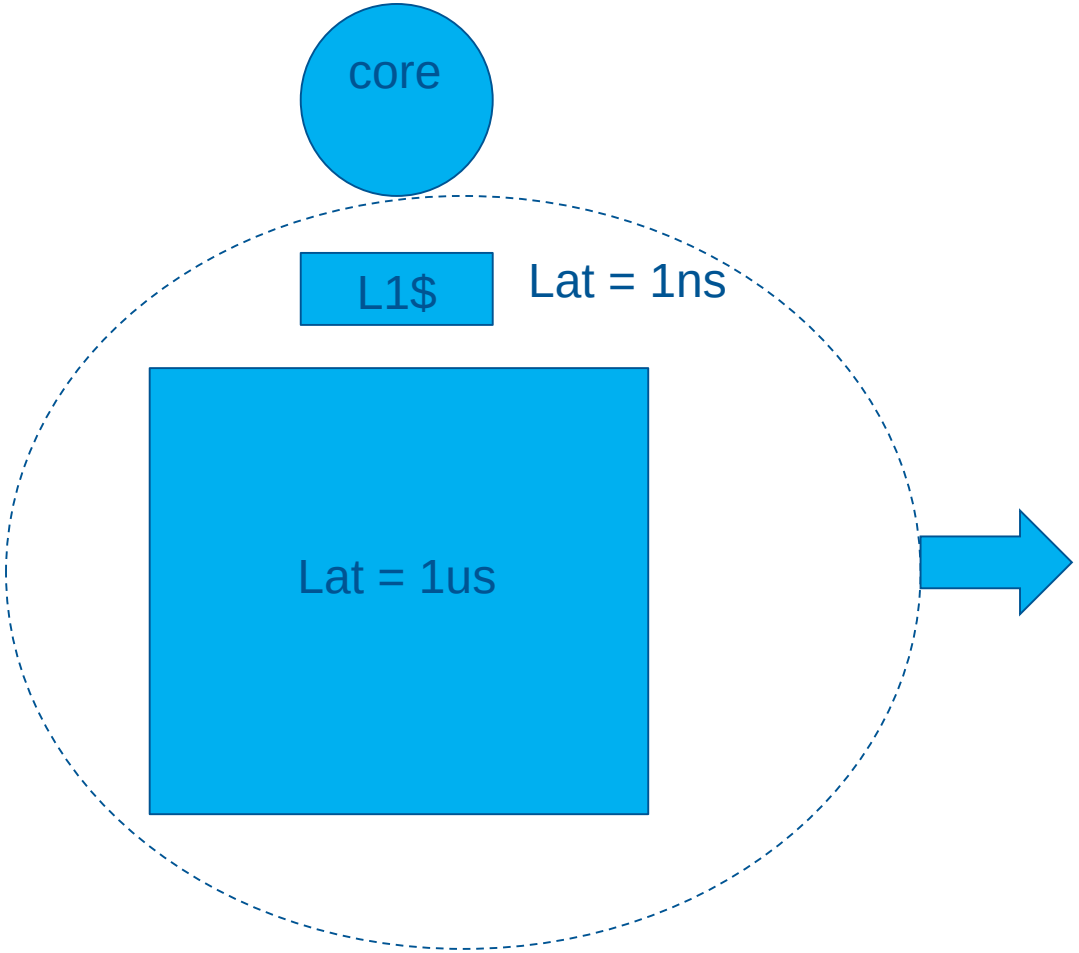


Assume 1 million requests

Total time with cache
= (HitRate*1ns + MissRate * 1us) * 1M

Conclusion: With increasing hit-rate, total access time approaches that of L1$

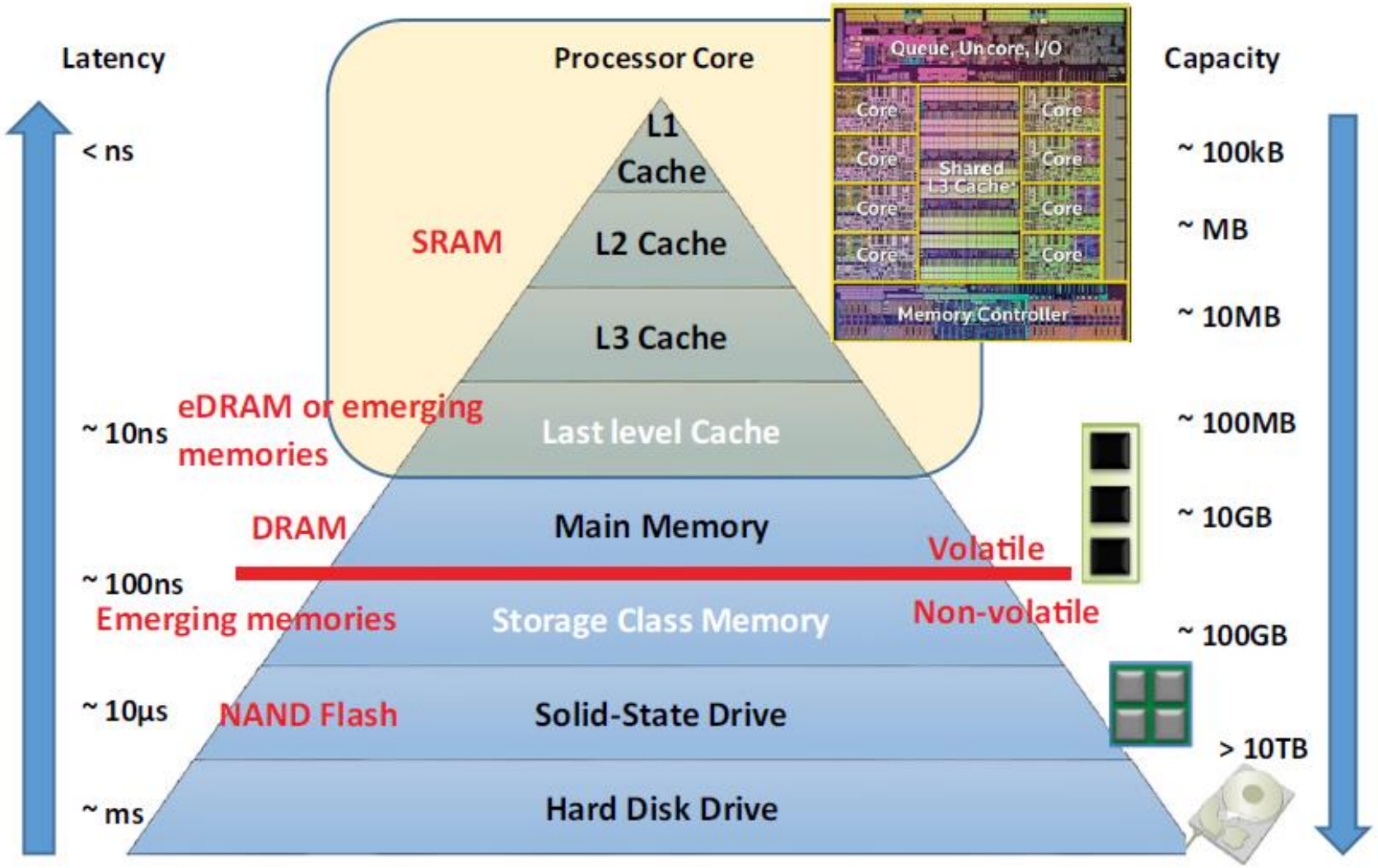
Why memory hierarchy

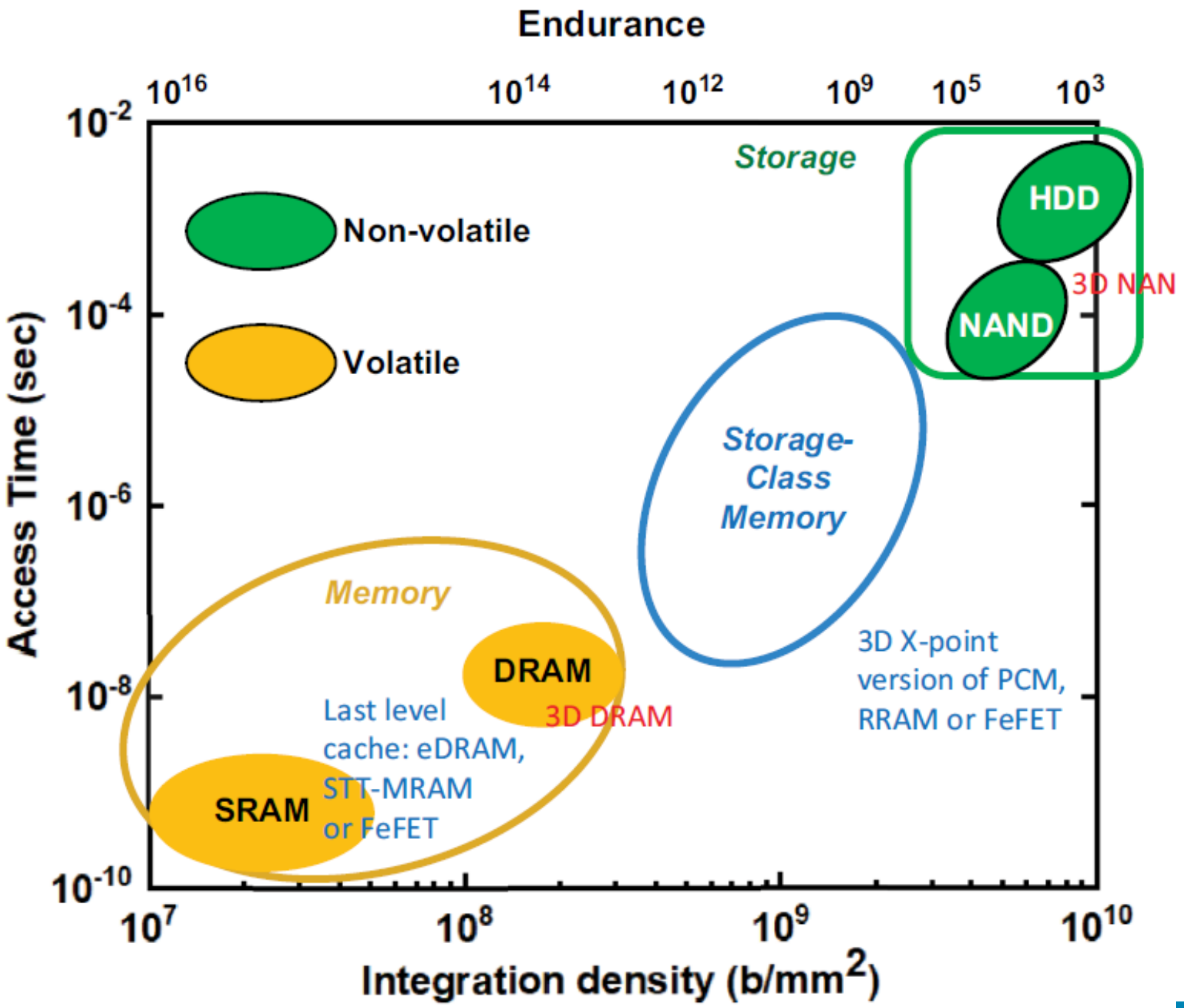


It appears to be a memory

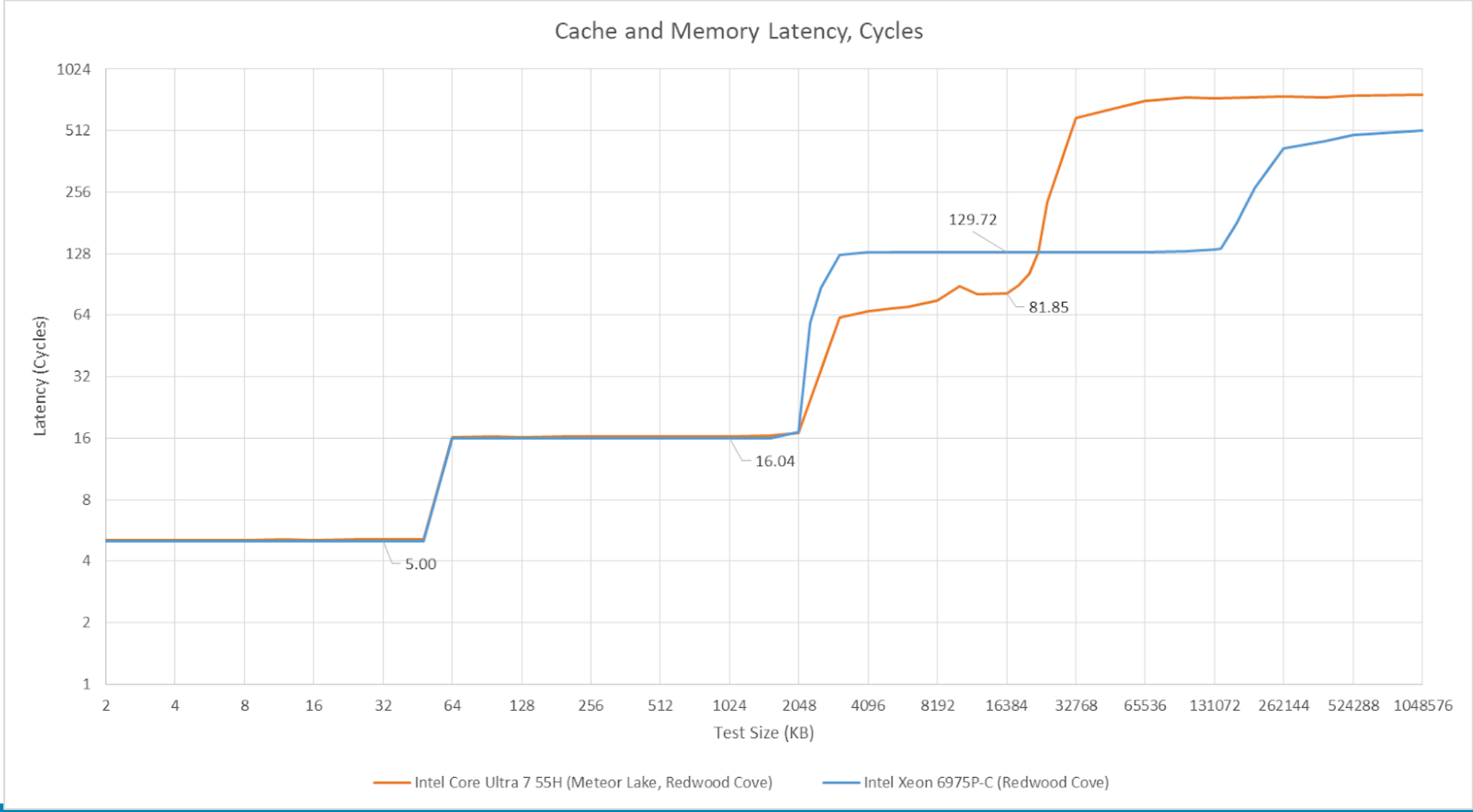
- As fast as L1 cache
- As large as main memory

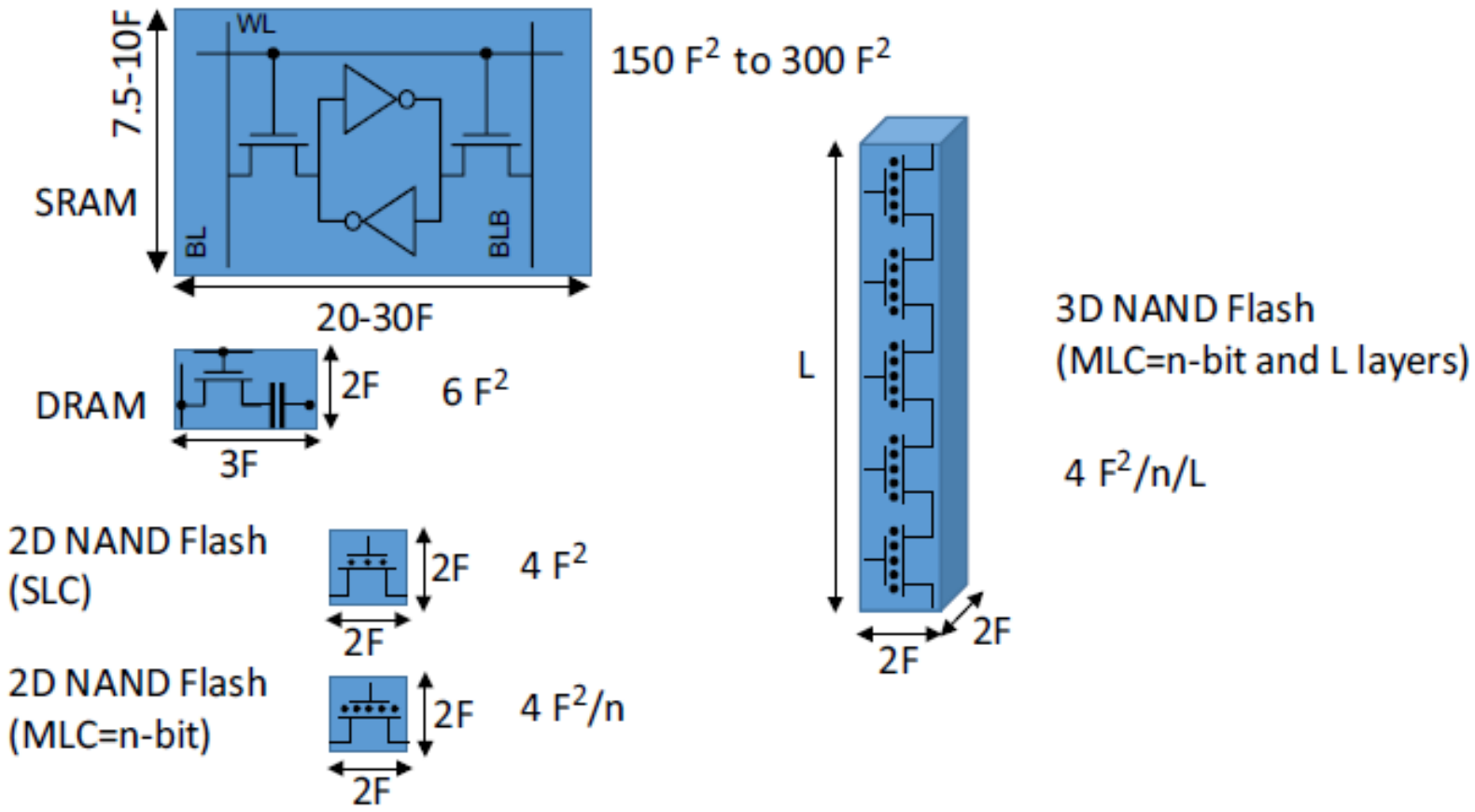
Approaches the characteristic of Ideal memory


Latency
< ns
~ 10ns
~ 100ns
~ 10µs
~ ms
Processor Core
L1 Cache
L2 Cache
L3 Cache
Last level Cache
Main Memory
Storage Class Memory
Solid-State Drive
Hard Disk Drive
SRAM
eDRAM or emerging memories
DRAM
Emerging memories
NAND Flash
Volatile
Non-volatile
Queue, Uncore, I/O
Core
Shared L3 Cache
Memory Controller
Capacity
~ 100kB
~ MB
~ 10MB
~ 100MB
~ 10GB
~ 100GB
> 10TB


Endurance
10^{16}
10^{14}
10^{12}
10^{9}
10^{5}
10^{3}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
Access Time (sec)
Storage
HDD
NAND
3D NAN
Non-volatile
Volatile
Storage-Class Memory
Memory
DRAM
3D DRAM
Last level cache: eDRAM, STT-MRAM or FeFET
SRAM
3D X-point version of PCM, RRAM or FeFET
10^{7}
10^{8}
10^{9}
10^{10}
Integration density (b/mm^2)

Latency of Meteor Lake: L1/L2/L3 = 5/16/130 cycles
Latency of Redwood Cove: L1/L2/L3 = 5/16/82 cycles



Cache Level	Xeon E3-1285 v4, Broadwell	Core i7-4770, Haswell	AMD Ryzen 9 7950X3D, VCache Die	Nvidia GH200
L1 Data	32 KB 8-way, 4 cycles	32 KB 8-way, 4 cycles	32 KB, 4 cycles	64 KB, 4 cycles
L2	256 KB 8-way, 12 cycles	256 KB 8-way, 12 cycles	1 MB 16-way, 14 cycles	1 MB, 11 cycles
L3	6 MB 12-way, ~40 cycles 10.42 ns	8 MB 16-way, ~38 cycles 9.77 ns	96 MB 16-way, ~54 cycles 10.63 ns	114 MB, >125 cycles 38.23 ns
L4	128 MB 16-way, ~139 cycles 36.6 ns	N/A	N/A	N/A

Out of course